AI-Powered Banking Support System

1. Executive Summary

The AI-Powered Banking Support System is designed to provide a secure, grounded, and efficient AI-driven assistant for banking operations. The system combines Retrieval-Augmented Generation (RAG) with secure backend integration to deliver accurate policy responses and controlled access to live banking data.

The solution emphasizes:

- Factual accuracy through document retrieval
- Secure backend integration
- Controlled AI behavior through guardrails
- Observability and cost optimization
- Enterprise-ready architecture design

This document outlines the structured use cases implemented as part of the system.

USECASE 1

Foundation & Core Retrieval for Banking Policies

Objective

Establish a strong foundation for building a grounded AI system capable of retrieving and answering policy-related banking queries using semantic search.

Business Context

Banks maintain extensive policy documents covering:

- Loan eligibility rules
- Interest rate guidelines
- Credit card terms
- EMI calculation frameworks

Manually searching these documents is inefficient. The objective is to enable semantic search over official policy documents while minimizing hallucination risks.

Functional Scope

This use case covers:

- Understanding LLM behavior and probabilistic nature
- Identifying hallucination and stale knowledge risks
- Designing a document ingestion pipeline
- Generating embeddings for semantic similarity
- Implementing vector database storage
- Comparing retrieval approaches (keyword vs semantic vs hybrid)

System Responsibilities

The AI system must:

- Retrieve policy data only from verified documents
- Avoid unsupported financial advice
- Use semantic similarity instead of keyword matching
- Apply chunking strategies for efficient retrieval
- Set similarity thresholds to filter weak matches

Implementation Approach

1. Policy documents are parsed and cleaned.
2. Documents are divided into structured chunks.
3. Embeddings are generated for each chunk.
4. Chunks and embeddings are stored in a vector database.
5. Similarity search retrieves relevant policy sections.

Deliverables

- Document ingestion pipeline
- Chunking configuration design

- Embedding generation module
- Vector database schema
- Retrieval comparison summary
- Hallucination risk analysis document

USECASE 2

End-to-End RAG Banking Assistant

Objective

Build a fully functional RAG-based banking assistant that retrieves policy information, generates grounded responses with citations, and uses LangSmith for tracing, debugging, and evaluation.

Business Context

After enabling retrieval, the system must generate responses that:

- Are grounded in retrieved policy documents
- Provide clear and reliable citations
- Prevent malicious, unsafe, or unsupported queries
- Are continuously improved using LangSmith traces and evaluation metrics

Functional Scope

This use case includes:

- LangChain-based embedding + retrieval workflows
- Prompt and response template construction
- Context-aware, citation-supported answer generation
- LangSmith-powered tracing & debugging
- Faithfulness, relevance, and hallucination detection
- Prompt-injection protection
- Safe fallback response handling

System Responsibilities

The AI must:

- Retrieve top-k relevant chunks
- Pass context + metadata to LLM
- Generate grounded, citation-based answers
- Detect & reduce hallucinations using LangSmith traces
- Block unsafe or injected prompts
- Provide fallback responses when retrieval is weak

Implementation Approach

1. Integrate retriever with prompt templates.
2. Pass user query + retrieved context to the LLM.
3. Attach document metadata to generate citation-enabled outputs.
4. Use LangSmith tracing to debug retrieval, prompts, and output quality.
5. Evaluate response faithfulness, relevance, latency, token usage using LangSmith.
6. Apply guardrails to reject unsafe inputs.

Deliverables

- Functional RAG pipeline with LangSmith integrated
- Prompt + retriever configuration
- Citation-enabled responses
- Guardrail and fallback logic
- LangSmith trace reports (retrieval, prompts, errors)
- Short evaluation summary (faithfulness, relevance, latency)

USECASE 3

Secure Banking Data Integration

Objective

Enable secure access to live banking data using controlled tool calling and authentication.

Business Context

Customers may request:

- Account balance
- Transaction history
- Loan outstanding details

This information must be retrieved securely and not generated by the AI model.

Functional Scope

- API layer integration
- JWT-based authentication
- Tool calling logic
- Secure backend data retrieval
- Masked response formatting

System Responsibilities

The AI must:

- Validate user authentication
- Call backend services securely
- Prevent unauthorized access
- Mask sensitive information
- Return structured JSON responses

Implementation Approach

1. Authenticate request using token validation.
2. Determine if tool calling is required.
3. Call backend banking service.
4. Fetch data from relational database.
5. Mask sensitive fields before response.

Deliverables

- Secure API endpoint
- JWT validation workflow
- Tool-calling integration module

- Structured response examples
- Security validation checklist

USECASE 4

Intelligence Maturity & Optimization

Objective

Improve system maturity through performance tracking, cost control, and observability.

Business Context

As system usage increases, cost and performance optimization become critical. The system must maintain response quality while controlling operational expense.

Functional Scope

- Token usage tracking
- Latency monitoring
- Context optimization
- Caching strategy
- Cost estimation
- Agent reasoning awareness (conceptual)

System Responsibilities

The AI must:

- Track token consumption per request
- Measure response latency
- Optimize context size
- Reduce unnecessary API calls
- Estimate cost per query

Implementation Approach

1. Add logging for token usage and latency.
2. Implement caching for frequent queries.

3. Tune similarity threshold.
4. Analyze cost vs performance trade-offs.

Deliverables

- Observability metrics report
- Token usage summary
- Cost estimation document
- Performance optimization summary
- Agent reasoning concept note.

USECASE 5

Final Integrated Banking RAG System

Objective

Deliver a complete, integrated AI-powered banking support system demonstrating architectural maturity.

Business Context

The organization requires a cohesive solution that integrates:

- Policy-based RAG
- Secure backend integration
- Guardrails
- Performance optimization
- Observability

Functional Scope

- End-to-end workflow integration
- System validation
- Documentation finalization
- Architecture review preparation

System Responsibilities

The AI system must:

- Provide grounded policy responses
- Securely retrieve live banking data
- Maintain performance and observability
- Enforce authentication and guardrails

Implementation Approach

1. Integrate RAG and backend services.
2. Validate full workflow.
3. Prepare system documentation.
4. Demonstrate complete functionality.

Deliverables

- Fully functional AI Banking Support System
- HLD and LLD documentation
- Architecture diagrams
- RAG evaluation summary
- Performance & observability summary
- Final presentation

USECASE 6 (With CI/CD)

Final Integrated Banking RAG System (with CI/CD)

Objective

Deliver a fully integrated, production-ready Banking RAG System that supports secure data retrieval, grounded responses, observability, compliance, and automated CI/CD deployment.

Business Context

After building individual components (retrieval, evaluation, guardrails), the final step is to assemble them into a complete, secure, auditable system that:

- Generates accurate and grounded responses

- Retrieves live backend data safely
- Protects sensitive financial information
- Ensures reliable releases through CI/CD workflows
- Maintains observability for debugging and compliance

Functional Scope

This use case includes:

- End-to-end RAG + tool-calling integration
- Secure backend API access (JWT, RBAC)
- PII masking & structured logging
- Observability (metrics, traces, latency, token usage)
- LangSmith-powered trace analysis & evaluation
- CI/CD pipeline execution with automated checks
- Deployment reliability with rollback strategies

System Responsibilities

The AI system must:

- Retrieve live and relevant banking data securely
- Produce grounded, accurate, citation-supported answers
- Mask sensitive data (PII, financial identifiers)
- Track performance (latency, tokens, retrieval hit-rate)
- Run evaluation and security checks in CI/CD
- Approve deployment only when tests pass

Implementation Approach

1. Build integrated RAG + tool-calling workflow.
2. Apply JWT-based backend authentication.
3. Implement masking, structured logs, and secure logging flow.
4. Add observability hooks (metrics + traces).
5. Use LangSmith for tracing, debugging, and evaluation of responses.

6. Create CI/CD pipeline that runs:
    - Retrieval & API tests
    - LangSmith evaluation stage
    - PII masking + security scans
    - Automated deployment on success

Deliverables

- Fully functional AI Banking RAG System
- Architecture documentation & diagrams
- Secure backend integration design
- Evaluation summary (accuracy, retrieval quality, latency)
- Performance & PII-masking report
- Operational CI/CD pipeline (GitHub Actions / Azure DevOps) with automated validation + deployment

Conclusion

The AI-Powered Banking Support System demonstrates a structured approach to building grounded, secure, and optimized AI applications. The solution emphasizes responsible AI practices, secure data access, and production-ready architecture principles.